Unidad 2 — JavaScript avanzado y herramientas del frontend

En la Unidad 1 aprendiste los fundamentos de JavaScript: variables, tipos, operadores, condicionales, bucles y funciones. En esta Unidad 2 damos un salto cualitativo hacia el JavaScript que se escribe a diario en proyectos reales. Trabajaremos en profundidad con **arrays** y **objetos**, entenderemos la diferencia crucial entre **referencia y valor**, desentrañaremos conceptos que suelen confundir (`this`, *closures*, prototipos) y dedicaremos una parte central a la **programación asíncrona**: el *event loop*, las promesas y `async/await`. Todo ello con sintaxis moderna (ECMAScript 2025/2026) y un fuerte énfasis en la **inmutabilidad** y las buenas prácticas actuales.

> 📌 Nota: Todos los ejemplos usan sintaxis moderna. Asume `'use strict'` o módulos ES (que ya son *strict mode* por defecto).

Índice

1. Arrays en profundidad
2. Objetos en profundidad
3. Referencia vs valor; copia superficial vs profunda
4. `this`, `call`, `apply`, `bind`
5. Closures y patrones
6. Prototipos y clases ES
7. `Map` y `Set`
8. Programación asíncrona: el *event loop*, promesas y `async/await`
9. La API `fetch`
10. Módulos ES
11. Manejo de errores
12. JSON
13. Web APIs del navegador (BOM)
14. Herramientas modernas: npm, módulos y Vite
15. Introducción a TypeScript

16. Buenas prácticas modernas
17. Ejercicios propuestos
18. Recursos y referencias

Arrays en profundidad

Un array es una colección ordenada e indexada (desde `0`) de valores. En JavaScript los arrays son objetos dinámicos: pueden crecer, encoger y contener tipos mezclados.

```
const numeros = [10, 20, 30];
const mezcla = [1, "dos", true, null, { id: 3 }];
const vacio = [];
const conLongitud = Array.from({ length: 3 }, (_, i) => i * 2); // [0, 2, 4]
```

Acceso a elementos: el método `at()`

El acceso clásico es por índice (`numeros[0]`), pero el método moderno **`at()`** admite índices negativos para contar desde el final, lo que evita la incómoda expresión `arr[arr.length - 1]`.

```
const letras = ["a", "b", "c", "d"];
console.log(letras.at(0));   // "a"
console.log(letras.at(-1));  // "d" (último elemento)
console.log(letras.at(-2));  // "c"
```

Métodos de mutación vs métodos inmutables

Esta distinción es de las más importantes de la unidad. Los **métodos de mutación** modifican el array original (efecto secundario). Los **métodos inmutables** devuelven un *nuevo* array y dejan el original intacto.

Mutación (modifica original)	Equivalente inmutable (ES2023+)
`sort()`	`toSorted()`
`reverse()`	`toReversed()`
`splice()`	`toSpliced()`
`arr[i] = x`	`with(i, x)`

```
// Métodos de mutación: alteran el array original
const pila = [1, 2, 3];
pila.push(4);        // añade al final → [1, 2, 3, 4]
pila.pop();          // elimina el último → [1, 2, 3]
pila.unshift(0);     // añade al inicio → [0, 1, 2, 3]
pila.shift();        // elimina el primero → [1, 2, 3]
pila.splice(1, 1, 9); // desde índice 1, borra 1 e inserta 9 → [1, 9, 3]
```

```
// Métodos inmutables: el original NO cambia
const original = [3, 1, 2];

const ordenado = original.toSorted((a, b) => a - b); // [1, 2, 3]
const invertido = original.toReversed();             // [2, 1, 3]
const cambiado = original.with(0, 99);               // [99, 1, 2]

console.log("ordenado:", ordenado);
console.log("original:", original); // [3, 1, 2]  ← intacto

// Ahora compáralo con la versión que muta:
const otro = [3, 1, 2];
otro.sort((a, b) => a - b);
console.log("tras sort(), el original:", otro); // ← ya no es [3, 1, 2]
```

> ⚠️ Cuidado: `sort()` y `reverse()` mutan el array. Además, `sort()` sin función comparadora ordena como **texto**: `[10, 2, 1].sort()` da `[1, 10, 2]`. Para números usa siempre `sort((a, b) => a - b)`.

> 💡 Consejo: Prefiere los métodos inmutables (`toSorted`, `toReversed`, `with`) salvo que tengas una razón concreta para mutar. El código inmutable es más predecible y encaja mejor con frameworks como React.

Iteración y transformación funcional

Los métodos de orden superior reciben una función y son el corazón del JavaScript moderno. Todos (excepto `forEach`) devuelven un valor nuevo sin mutar el original.

```
const productos = [
  { nombre: "Teclado", precio: 25, stock: 0 },
  { nombre: "Ratón",   precio: 15, stock: 12 },
  { nombre: "Monitor", precio: 180, stock: 4 },
];

// map: transforma cada elemento → nuevo array de misma longitud
const nombres = productos.map(p => p.nombre);
// ["Teclado", "Ratón", "Monitor"]

// filter: conserva los que cumplen la condición
const disponibles = productos.filter(p => p.stock > 0);
// [Ratón, Monitor]

// reduce: acumula en un único valor
const valorTotal = productos.reduce((acc, p) => acc + p.precio * p.stock, 0);
// 0 + 180 + 720 = 900  (0*25 + 12*15 + 4*180)

// find / findIndex: primer elemento (o índice) que cumple
const monitor = productos.find(p => p.precio > 100); // objeto Monitor

// some / every: devuelven booleanos
const hayCaros = productos.some(p => p.precio > 100);   // true
const todosBaratos = productos.every(p => p.precio < 50); // false

// flatMap: map + aplanar un nivel
const etiquetas = productos.flatMap(p => [p.nombre, p.precio]);
// ["Teclado", 25, "Ratón", 15, "Monitor", 180]

console.log("nombres:", nombres);
console.log("disponibles:", disponibles.map(p => p.nombre));
console.log("valor total del stock:", valorTotal);
console.log("¿hay algo por encima de 100 €?:", hayCaros);
```

Método	¿Muta?	Devuelve	Uso típico
`map`	No	Array nuevo	Transformar cada elemento
`filter`	No	Array nuevo	Seleccionar elementos
`reduce`	No	Valor acumulado	Sumar, agrupar, combinar
`find`	No	Elemento o `undefined`	Buscar uno
`some` / `every`	No	`boolean`	Validar condiciones
`flatMap`	No	Array aplanado	Transformar y expandir
`forEach`	No (pero suele tener efectos)	`undefined`	Ejecutar acción por elemento

> 💡 Consejo: Encadena métodos para *pipelines* legibles: `productos.filter(p => p.stock > 0).map(p => p.nombre).toSorted()`.

Objetos en profundidad

Un objeto es una colección de pares **clave-valor**. Las claves son cadenas (o `Symbol`) y los valores, cualquier tipo, incluyendo funciones (que entonces llamamos **métodos**).

```
const usuario = {
  nombre: "Lucía",
  edad: 28,
  saludar() {                 // shorthand de método
    return `Hola, soy ${this.nombre}`;
  },
};
console.log(usuario.nombre);     // acceso con punto
console.log(usuario["edad"]);    // acceso con corchetes
```

> 📌 Nota: Recuerda el encadenamiento opcional `?.` y el *nullish coalescing* `??` de la Unidad 1: son especialmente útiles al navegar objetos anidados. Por ejemplo: `const ciudad = usuario.direccion?.ciudad ?? "desconocida";`.

Shorthand y propiedades computadas

```
const nombre = "Ana";
const edad = 30;

// Shorthand: si la variable se llama igual que la clave
const persona = { nombre, edad }; // { nombre: "Ana", edad: 30 }

// Computed properties: clave dinámica entre corchetes
const clave = "rol";
const config = { [clave]: "admin", [`nivel_${edad}`]: true };
// { rol: "admin", nivel_30: true }
```

Desestructuración (*destructuring*)

Extrae valores de objetos y arrays a variables de forma concisa. Admite renombrado, valores por defecto y anidación.

```
// De objetos
const { nombre: n, edad: e = 0, ciudad = "Madrid" } = persona;
// n = "Ana", e = 30, ciudad = "Madrid" (valor por defecto)

// De arrays (por posición)
const [primero, , tercero] = ["a", "b", "c"];
// primero = "a", tercero = "c"

// En parámetros de función (muy común)
function crearTarjeta({ titulo, color = "azul" }) {
  return `${titulo} (${color})`;
}
crearTarjeta({ titulo: "Inicio" }); // "Inicio (azul)"
```

Spread y rest (`...`)

El operador `...` **expande** (spread) o **agrupa** (rest) según el contexto.

```
// Spread: copiar y combinar
const base = { tema: "oscuro", idioma: "es" };
const ampliado = { ...base, idioma: "en", fuente: 14 };
// { tema: "oscuro", idioma: "en", fuente: 14 }  ← idioma se sobreescribe

const a = [1, 2], b = [3, 4];
const unidos = [...a, ...b]; // [1, 2, 3, 4]

// Rest: recoger el resto
const { tema, ...otras } = ampliado; // tema="oscuro", otras={idioma, fuente}

function suma(...numeros) {           // rest en parámetros
  return numeros.reduce((t, n) => t + n, 0);
}
suma(1, 2, 3, 4); // 10
```

Métodos estáticos de `Object`

```
const punto = { x: 1, y: 2 };

Object.keys(punto);    // ["x", "y"]
Object.values(punto);  // [1, 2]
Object.entries(punto); // [["x", 1], ["y", 2]]

// Reconstruir desde entries (útil tras filtrar/transformar)
Object.fromEntries([["a", 1], ["b", 2]]); // { a: 1, b: 2 }

// Combinar (muta el primer argumento) — preferible usar spread
Object.assign({}, base, { idioma: "fr" });

// Congelar: impide añadir, borrar o modificar propiedades
const constante = Object.freeze({ PI: 3.1416 });
constante.PI = 3; // ✗ TypeError: el objeto está congelado (en strict mode)
```

`Object.groupBy` (ES2024)

Agrupa los elementos de un iterable en un objeto según el valor devuelto por una función. Sustituye con elegancia a soluciones manuales con `reduce`.

```
const inventario = [
  { nombre: "manzana", tipo: "fruta" },
  { nombre: "zanahoria", tipo: "verdura" },
  { nombre: "plátano", tipo: "fruta" },
];

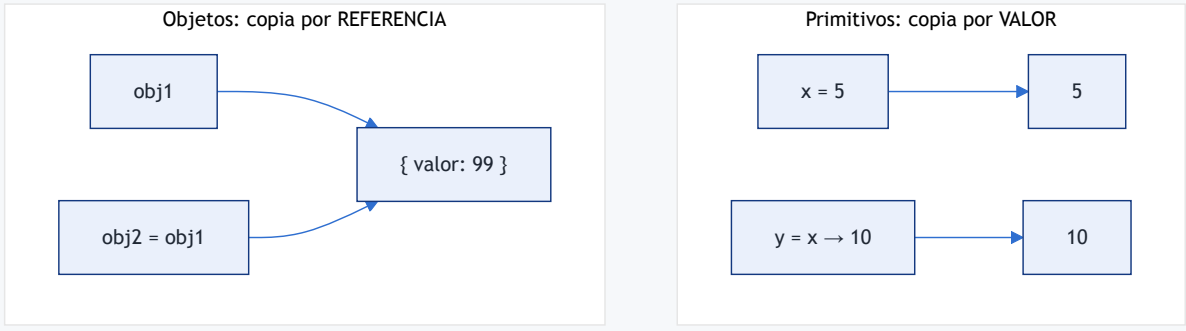
const porTipo = Object.groupBy(inventario, item => item.tipo);
// {
//   fruta:   [{manzana}, {plátano}],
//   verdura: [{zanahoria}]
// }

console.log(porTipo);
console.log("frutas:", porTipo.fruta.map(f => f.nombre));
```

Existe también `Map.groupBy`, que hace lo mismo pero devuelve un `Map`: útil cuando las claves de agrupación no son strings (por ejemplo, objetos).

Referencia vs valor; copia superficial vs profunda

Los **tipos primitivos** (`number`, `string`, `boolean`, `null`, `undefined`, `symbol`, `bigint`) se copian **por valor**. Los **objetos** (incluidos arrays y funciones) se manejan **por referencia**: la variable guarda una "dirección", no el dato.



A la izquierda, `y = x` copia el número: son dos cajas distintas y cambiar una no toca la otra. A la derecha, `obj2 = obj1` copia **la flecha**, no la caja: hay un único objeto con dos nombres, así que modificarlo por cualquiera de los dos se ve por el otro.

```
// Por valor: copia independiente
let x = 5;
let y = x;
y = 10;
console.log("x:", x); // 5  ← no afectado

// Por referencia: ambas variables apuntan al mismo objeto
const obj1 = { valor: 1 };
const obj2 = obj1;
obj2.valor = 99;
console.log("obj1.valor:", obj1.valor); // 99  ← ¡afectado!
```

Copia superficial (*shallow*)

`{ ...obj }` y `Object.assign` copian solo el primer nivel. Los objetos anidados siguen compartiendo referencia.

```
const original = { nombre: "Eva", config: { tema: "claro" } };
const copia = { ...original };

copia.nombre = "Mar";            // afecta solo a copia (primer nivel)
copia.config.tema = "oscuro";    // ¡afecta también a original! (anidado)

console.log(original.config.tema); // "oscuro"
```

Copia profunda (*deep*) con `structuredClone`

`structuredClone()` (estándar moderno, nativo en navegadores y Node) clona en profundidad, incluyendo objetos anidados, `Map`, `Set`, `Date`, etc.

```
const original = { config: { tema: "claro" }, fecha: new Date() };
const clon = structuredClone(original);

clon.config.tema = "oscuro";
console.log(original.config.tema); // "claro"  ← independiente
```

> ⚠️ Cuidado: El viejo truco `JSON.parse(JSON.stringify(obj))` funciona para datos simples, pero pierde funciones, `undefined`, `Date` (se convierte en string) y falla con referencias circulares. Usa `structuredClone` para copias profundas fiables.

`this`, `call`, `apply`, `bind`

`this` es una referencia que **se determina al llamar a la función**, no al definirla. Su valor depende de *cómo* se invoca.

```
const coche = {
  marca: "Tesla",
  describir() {
    return `Coche: ${this.marca}`; // this = coche
  },
};
coche.describir(); // "Coche: Tesla"  (this es el objeto a la izquierda del punto)
```

Pérdida de contexto

El caso clásico: al **extraer** un método a una variable (o pasarlo como *callback*), se pierde el objeto "a la izquierda del punto" y, con él, el `this`.

```
const f = coche.describir;
f(); // ✗ TypeError: this es undefined en strict mode
```

> ⚠️ Cuidado: esta pérdida de contexto es la motivación real de `bind` (más abajo): `const g = coche.describir.bind(coche); g(); // "Coche: Tesla"`.

Las **funciones flecha** no tienen su propio `this`: heredan el del contexto donde se definieron. Esto las hace ideales para *callbacks* dentro de métodos.

```
const temporizador = {
  segundos: 0,
  iniciar() {
    setInterval(() => {
      this.segundos++; // this = temporizador (heredado), no global
    }, 1000);
  },
};
```

`call`, `apply` y `bind`

Permiten fijar `this` manualmente.

```
function presentar(saludo, signo) {
  return `${saludo}, soy ${this.nombre}${signo}`;
}
const ana = { nombre: "Ana" };

presentar.call(ana, "Hola", "!");      // argumentos sueltos → "Hola, soy Ana!"
presentar.apply(ana, ["Hola", "!"]);   // argumentos en array → idéntico
const saludarAna = presentar.bind(ana); // devuelve función con this fijado
saludarAna("Buenas", ".");             // "Buenas, soy Ana."
```

> 📌 Nota: `call` / `apply` ejecutan ya; `bind` devuelve una *nueva* función que recordará su `this` para invocarla después.

Closures y patrones

Un **closure** (clausura) es la capacidad de una función de "recordar" las variables del ámbito donde fue creada, incluso después de que ese ámbito haya terminado. Es la base de muchos patrones potentes.

```
function crearContador() {
  let cuenta = 0;            // variable "privada" capturada por el closure
  return {
    incrementar() { return ++cuenta; },
    valor() { return cuenta; },
  };
}

const contador = crearContador();
contador.incrementar(); // 1
contador.incrementar(); // 2

console.log("valor:", contador.valor());          // 2
console.log("¿se ve la variable?:", contador.cuenta); // undefined ← encapsulación

// Cada llamada a la factory crea su propio ámbito independiente
const otro = crearContador();
console.log("otro contador:", otro.valor()); // 0, no comparte estado
```

Patrón *factory* (fábrica)

```
function crearUsuario(nombre, rol) {
  return {
    nombre,
    rol,
    puedeEditar: () => rol === "admin",
  };
}
const editor = crearUsuario("Leo", "admin");
editor.puedeEditar(); // true
```

Patrón módulo

```
const carrito = (function () {
  const items = [];                    // estado privado
  return {
    add(item) { items.push(item); },
    total() { return items.length; },
  };
})();
carrito.add("libro");
carrito.total(); // 1
```

Prototipos y clases ES

Bajo el capó, JavaScript usa **herencia por prototipos**: cada objeto tiene un enlace interno a otro objeto (su prototipo) del que hereda propiedades y métodos. Las **clases** (`class`) son *azúcar sintáctico* sobre este modelo: más legibles, mismo motor.

```
class Animal {
  #energia = 100;          // campo privado (solo accesible dentro de la clase)
  static especies = 0;     // propiedad estática (de la clase, no de instancias)

  constructor(nombre) {
    this.nombre = nombre;
    Animal.especies++;
  }

  comer() {                // método de instancia (vive en el prototipo)
    this.#energia += 10;
    return `${this.nombre} come`;
  }

  get energia() { return this.#energia; }       // getter
  set energia(v) { this.#energia = Math.max(0, v); } // setter

  static crearAnonimo() { return new Animal("???"); } // método estático
}

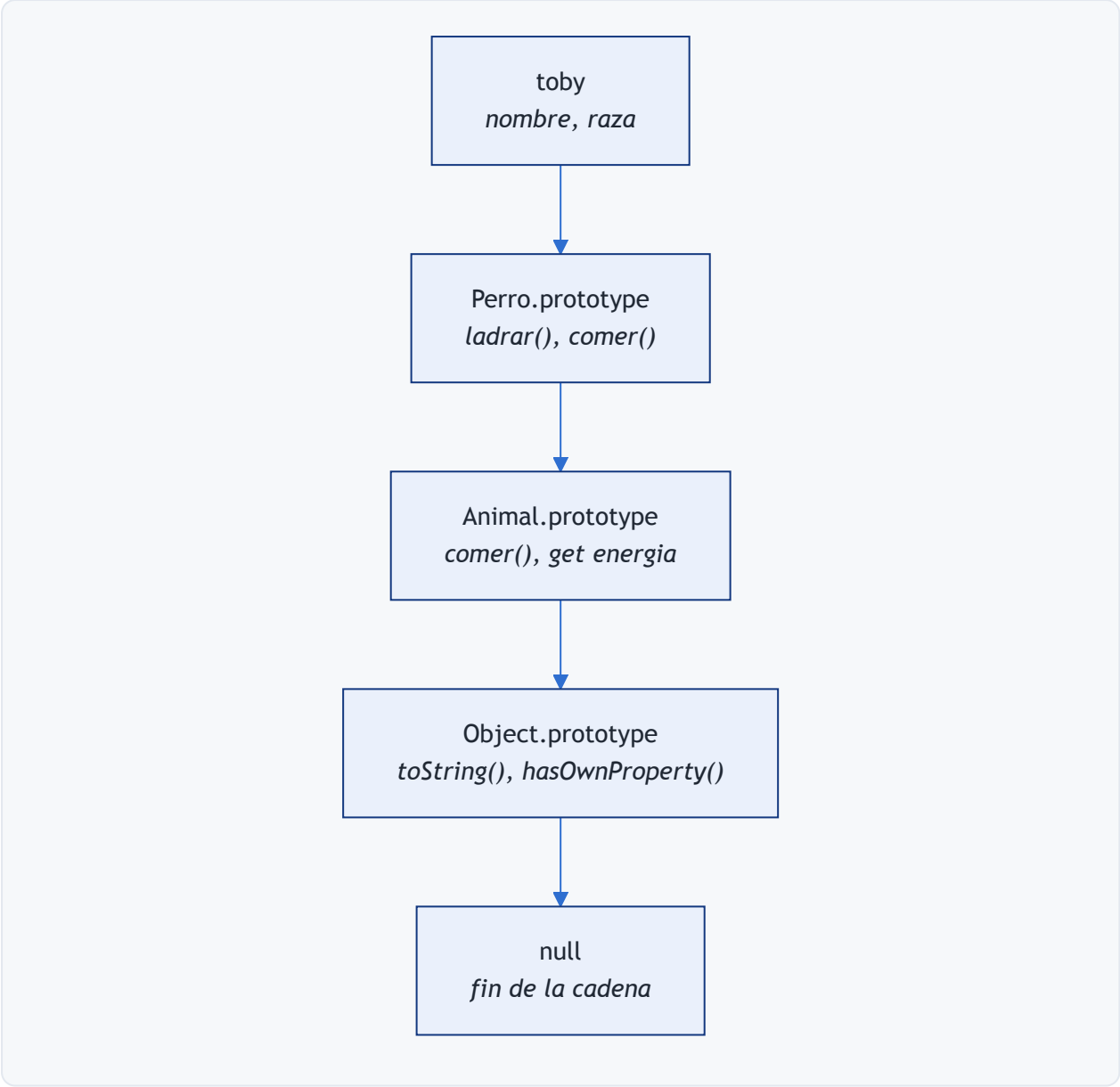
const gato = new Animal("Misi");
gato.comer();          // "Misi come"
gato.energia;          // 110  (usa el getter)
Animal.especies;       // 1
```

Herencia con `extends` y `super`

```
class Perro extends Animal {
  constructor(nombre, raza) {
    super(nombre);       // llama al constructor padre (obligatorio antes de this)
    this.raza = raza;
  }
  ladrar() { return `${this.nombre}: ¡Guau!`; }
  comer() {
    return super.comer() + " con ganas"; // extiende el método del padre
  }
}

const toby = new Perro("Toby", "Labrador");
toby.ladrar();  // "Toby: ¡Guau!"
toby.comer();   // "Toby come con ganas"
toby instanceof Animal; // true
```

📌 Nota: El modelo subyacente: `Object.getPrototypeOf(toby) === Perro.prototype` y `Object.getPrototypeOf(Perro.prototype) === Animal.prototype`. Cuando accedes a un método, JavaScript sube por esta **cadena de prototipos** hasta encontrarlo. La propiedad `__proto__` que quizá veas en código antiguo es **legado** y no debe usarse: emplea siempre `Object.getPrototypeOf()`.



Sigue la flecha para entender `toby.comer()`: no está en la instancia, se encuentra en `Perro.prototype` y ahí se detiene la búsqueda; como ese método llama a `super.comer()`, se salta un eslabón y ejecuta el de `Animal.prototype`. Si pidieras `toby.hasOwnProperty("raza")`, la búsqueda subiría hasta `Object.prototype`. Y si pidieras algo inexistente, llegaría a `null` y devolvería `undefined`.

`Map` y `Set`

Estructuras de datos modernas que complementan a objetos y arrays.

`Map`: pares clave-valor donde la clave puede ser *cualquier* tipo (no solo strings), mantiene el orden de inserción y tiene un `size` directo.

```
const cache = new Map();
cache.set("usuario", { id: 1 });
cache.set(42, "respuesta");        // clave numérica
cache.get("usuario");              // { id: 1 }
cache.has(42);                     // true
cache.size;                        // 2
for (const [clave, valor] of cache) { /* itera en orden */ }
```

`Set`: colección de valores **únicos** (sin duplicados).

```
const etiquetas = new Set(["js", "css", "js", "html"]);
etiquetas.size;        // 3  (el "js" duplicado se descarta)
etiquetas.add("react");
etiquetas.has("css");  // true

// Truco habitual: eliminar duplicados de un array
const sinDuplicados = [...new Set([1, 1, 2, 3, 3])]; // [1, 2, 3]

console.log("tamaño tras añadir 'react':", etiquetas.size); // 4: eran 3 + la nueva
console.log("sin duplicados:", sinDuplicados);
```

Además, `Set` incorpora métodos de composición al estilo de la teoría de conjuntos, ya estables en todos los navegadores (Baseline 2024–2025): `union()`, `intersection()` y `difference()`.

```
const frontend = new Set(["js", "css", "html"]);
const backend = new Set(["js", "sql"]);

console.log("unión:", frontend.union(backend));              // Set {"js", "css",
console.log("intersección:", frontend.intersection(backend)); // Set {"js"}
console.log("diferencia:", frontend.difference(backend));     // Set {"css", "html"
```

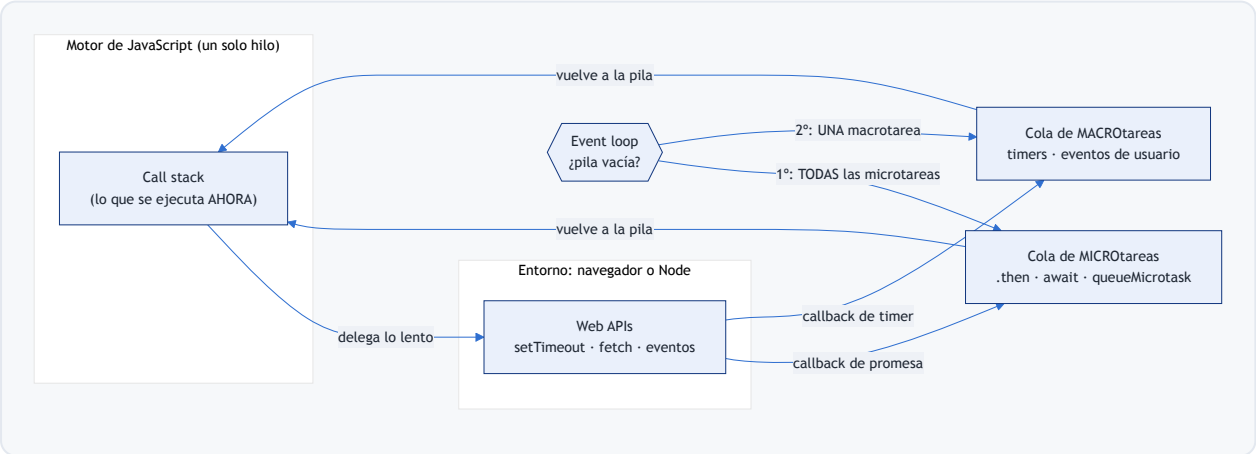
Usa...	cuando...
Objeto	claves string fijas y conocidas, datos estructurados
`Map`	claves dinámicas o de cualquier tipo, muchas altas/bajas, necesitas `size` /orden
Array	colección ordenada con posibles duplicados
`Set`	colección de valores únicos, comprobar pertenencia rápido

Programación asíncrona: el *event loop*, promesas y `async/await`

JavaScript es **monohilo** (*single-threaded*): ejecuta una sola cosa a la vez en su *call stack* (pila de llamadas). Para no bloquearse mientras espera operaciones lentas (red, temporizadores, ficheros), usa un modelo **asíncrono** coordinado por el **event loop**.

El event loop

1. El código síncrono se ejecuta en la **call stack**.
2. Las operaciones asíncronas (fetch, `setTimeout`) se delegan al entorno (navegador/Node).
3. Al completarse, sus *callbacks* se encolan: las **macrotareas** (timers, eventos) en la *task queue*; las **microtareas** (callbacks de promesas) en la *microtask queue*.
4. Cuando la call stack queda vacía, el event loop **vacía primero TODAS las microtareas** y luego toma una macrotarea.



El diagrama explica la regla que más cae en los exámenes y en las defensas: da igual que el `setTimeout` sea de `0` milisegundos, porque su callback va a una cola que solo se atiende

después de haber vaciado entera la de microtareas.

```
console.log("1");                          // síncrono
setTimeout(() => console.log("2"), 0);     // macrotarea
Promise.resolve().then(() => console.log("3")); // microtarea
console.log("4");                          // síncrono

// Predice el orden ANTES de pulsar Ejecutar.
```

> 💡 Consejo: Recuerda la regla de oro: las promesas (microtareas) tienen prioridad sobre `setTimeout` (macrotarea), aunque el timeout sea `0`.

> 🧪 Experimento: en el bloque de arriba pulsa **Editar** y añade un segundo `Promise.resolve().then(() => console.log("3 bis"))` justo después del primero. ¿Se cuela antes o después del `2`? Compruébalo y explica por qué.

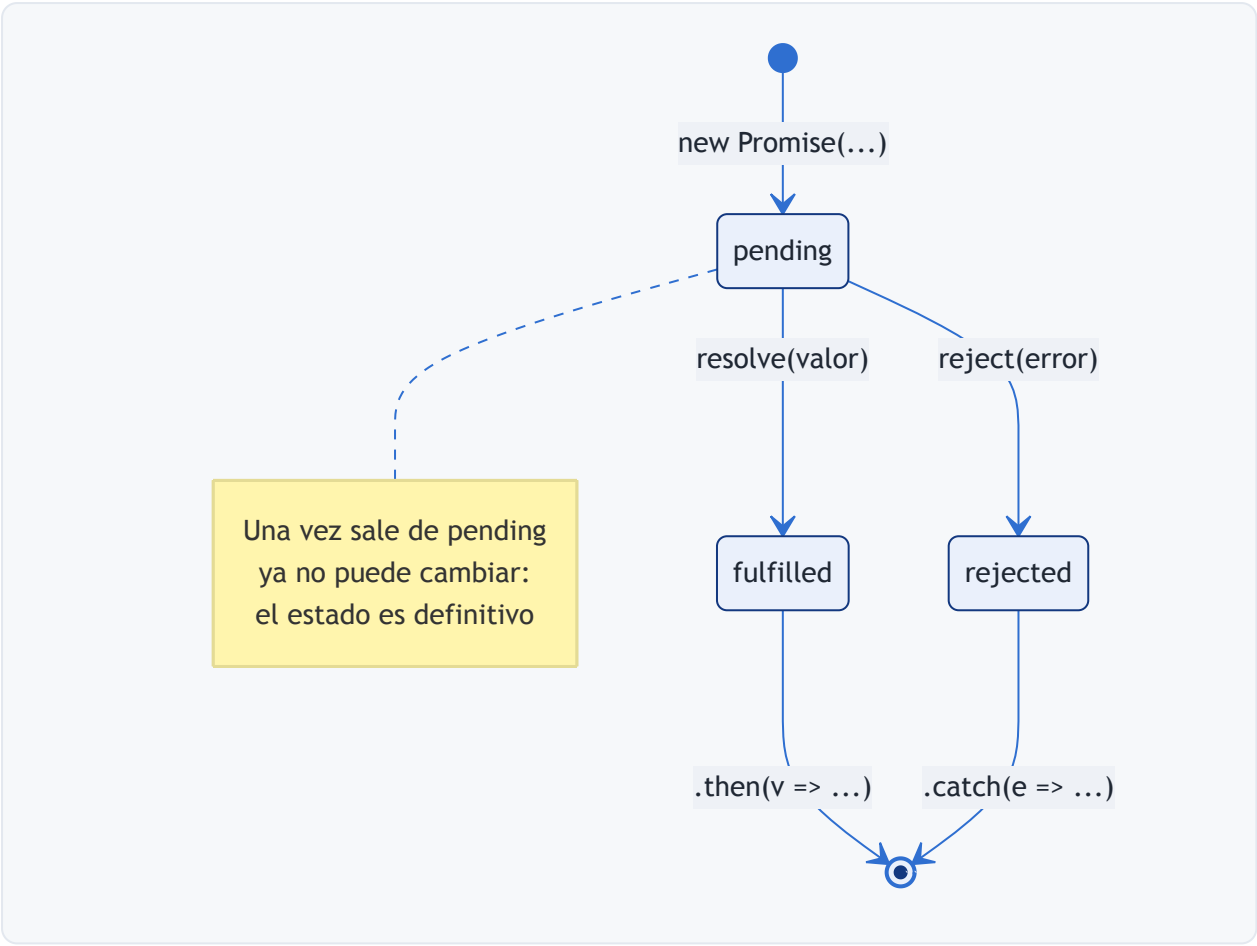
Callbacks y el "callback hell"

Antes de las promesas, la asincronía se encadenaba con callbacks anidados, generando código difícil de leer y mantener (la "pirámide de la perdición").

```
// ❌ Callback hell
obtenerUsuario(id, (usuario) => {
  obtenerPedidos(usuario, (pedidos) => {
    obtenerDetalle(pedidos[0], (detalle) => {
      console.log(detalle); // anidamiento creciente, errores difíciles
    });
  });
});
```

Promesas

Una **promesa** representa el resultado futuro de una operación asíncrona. Tiene tres estados: *pending* (pendiente), *fulfilled* (resuelta) o *rejected* (rechazada).



Dos consecuencias prácticas de que el estado sea **definitivo**: llamar a `resolve()` dos veces no hace nada la segunda, y una promesa ya resuelta que reciba un `.then()` más tarde ejecutará ese callback igualmente (en la siguiente microtarea), no se lo pierde.

```
const promesa = new Promise((resolve, reject) => {
  const exito = true;
  if (exito) resolve("¡Datos listos!");
  else reject(new Error("Algo falló"));
});

promesa
  .then(resultado => console.log(resultado)) // si resolve
  .catch(error => console.error(error))      // si reject
  .finally(() => console.log("Terminado"));  // siempre
```

Las promesas se **encadenan**, aplanando el callback hell:

```
obtenerUsuario(id)
  .then(usuario => obtenerPedidos(usuario))
  .then(pedidos => obtenerDetalle(pedidos[0]))
  .then(detalle => console.log(detalle))
  .catch(error => console.error("Error en la cadena:", error));
```

Combinadores de promesas

```
const p1 = Promise.resolve(1);
const p2 = Promise.resolve(2);

// all: espera a TODAS; falla si alguna falla
await Promise.all([p1, p2]);        // [1, 2]

// race: resuelve/rechaza con la PRIMERA que termine
await Promise.race([p1, p2]);       // 1

// allSettled: espera a todas, nunca falla; informa de cada resultado
const p3 = Promise.reject(new Error("fallo"));
await Promise.allSettled([p1, p3]);
// [{status:"fulfilled", value:1}, {status:"rejected", reason:Error}]

// any: la primera que se RESUELVA (ignora rechazos hasta que todas fallen)
await Promise.any([p3, p2]);        // 2
```

> ⚠️ Cuidado: creamos `p3` (la promesa rechazada) justo antes de usarla. Si una promesa se rechaza y pasan varios "turnos" sin que nadie maneje ese rechazo, el motor emite un aviso de *unhandled rejection*.

> 📌 Nota: el `await` a nivel superior de este ejemplo (*top-level await*) solo funciona en **módulos ES**; en un script clásico deberías envolverlo en una función `async`.

Combinador	Resultado
`all`	Todas OK → array; una falla → rechaza inmediatamente
`allSettled`	Siempre resuelve con el estado de cada una
`race`	La primera en terminar (resuelva o rechace)
`any`	La primera en **resolverse**

`async/await`

`async/await` es azúcar sintáctico sobre promesas que permite escribir código asíncrono con aspecto síncrono. Una función `async` siempre devuelve una promesa; `await` pausa hasta que la promesa se resuelve.

```
async function cargarDetalle(id) {
  try {
    const usuario = await obtenerUsuario(id);
    const pedidos = await obtenerPedidos(usuario);
    const detalle = await obtenerDetalle(pedidos[0]);
    return detalle;
  } catch (error) {
    console.error("Error:", error.message); // captura cualquier rechazo
  } finally {
    console.log("Carga finalizada");
  }
}
```

> 💡 Consejo: Si varias operaciones son **independientes**, no las pongas en `await` secuenciales (lento). Lánzalas en paralelo con `Promise.all`:
>
> ```
> const [a, b] = await Promise.all([cargarA(), cargarB()]);
> ```

Este es el error de rendimiento más común al empezar con `async/await`, y se ve mejor cronometrado que explicado:

```
// Simula tres peticiones que tardan 400 ms cada una
const tardar = (ms, valor) => new Promise(r => setTimeout(() => r(valor), ms));

// ❌ Secuencial: cada await espera al anterior
let t0 = performance.now();
const a1 = await tardar(400, "A");
const b1 = await tardar(400, "B");
const c1 = await tardar(400, "C");
console.log(`Secuencial: ${Math.round(performance.now() - t0)} ms`);

// ✅ En paralelo: las tres salen a la vez
t0 = performance.now();
const [a2, b2, c2] = await Promise.all([tardar(400, "A"), tardar(400, "B"), tardar
console.log(`Promise.all: ${Math.round(performance.now() - t0)} ms`);

console.log("Mismo resultado:", [a2, b2, c2].join(""));
```

> ⚠️ Cuidado: el paralelo solo vale si las operaciones son **independientes**. Si la segunda petición necesita un dato que devuelve la primera, no hay atajo: ahí el `await` secuencial es la respuesta correcta y hay que saber decirlo.

Los tres estados de una carga

Toda pantalla que pide datos tiene tres estados, y los tres se ven: **cargando**, **error** y **datos** (más el caso de datos vacíos). Es justo lo que se evalúa en la misión de esta unidad. Prueba a pulsar los dos botones:

```
<!doctype html>
<html lang="es">
<body style="font-family: system-ui; padding: 8px">
  <button id="ok">Cargar bien</button>
  <button id="mal">Provocar un error</button>
  <div id="salida" style="margin-top: 12px; min-height: 3em"></div>

  <script>
    const salida = document.getElementById("salida");

    // Simula una petición que tarda y que a veces falla
    const pedirDatos = (fallar) =>
      new Promise((resolve, reject) =>
        setTimeout(() => (fallar ? reject(new Error("HTTP 500")) : resolve(["Ana",
      );

    async function cargar(fallar) {
      salida.innerHTML = "<p>⏳ Cargando…</p>";          // 1. estado de carga
      try {
        const datos = await pedirDatos(fallar);
        salida.innerHTML = datos.length
          ? "<ul>" + datos.map((d) => `<li>${d}</li>`).join("") + "</ul>"  // 3. da
          : "<p>No hay resultados.</p>";                                    // 3b. v
      } catch (e) {
        salida.innerHTML = `<p style="color:#c00">⚠ ${e.message}. <button onclick="
      }
    }

    document.getElementById("ok").onclick = () => cargar(false);
    document.getElementById("mal").onclick = () => cargar(true);
  </script>
</body>
</html>
```

La API `fetch`

`fetch` realiza peticiones HTTP y devuelve una promesa que resuelve con un objeto `Response`. Lo ampliaremos en la Unidad 4; aquí vemos lo esencial.

```
async function obtenerUsuarios() {
  try {
    const respuesta = await fetch("https://api.example.com/usuarios");

    // fetch NO rechaza con errores HTTP (404, 500): hay que comprobar ok
    if (!respuesta.ok) {
      throw new Error(`HTTP ${respuesta.status}: ${respuesta.statusText}`);
    }

    const datos = await respuesta.json(); // parsea el cuerpo JSON
    return datos;
  } catch (error) {
    console.error("No se pudieron cargar los usuarios:", error.message);
    return [];
  }
}
```

⚠️ Cuidado: `fetch` solo rechaza la promesa ante fallos de **red**. Un `404` o `500` se considera respuesta válida: por eso debes comprobar siempre `respuesta.ok`.

📌 Nota: Las peticiones `fetch` se pueden **cancelar** con `AbortController` (o la forma abreviada `AbortSignal.timeout(ms)` para fijar un tiempo máximo). Lo veremos en detalle en la Unidad 4.

Módulos ES

Los **módulos ES** (ESM) permiten dividir el código en ficheros reutilizables con su propio ámbito. Se usan `export` para exponer e `import` para consumir.

```
// matematicas.js
export const PI = 3.1416;                // exportación con nombre
export function area(r) { return PI * r * r; }
export default function saludar() { ... }  // exportación por defecto (una por módu
```

```
// app.js
import saludar, { PI, area } from "./matematicas.js";
import { area as calcularArea } from "./matematicas.js"; // renombrado
import * as Mate from "./matematicas.js";                // todo como namespace

console.log(area(2), PI);
```

Import dinámico

`import()` carga un módulo bajo demanda (devuelve una promesa). Ideal para *code splitting* y cargar funcionalidad solo cuando se necesita.

```
async function cargarGraficos() {
  const modulo = await import("./graficos.js");
  modulo.dibujar();
}
```

> 📌 Nota: En HTML, los módulos se cargan con `<script type="module" src="app.js"></script>`. Los módulos son *strict mode* por defecto y diferidos automáticamente.

Manejo de errores

Usa `throw` para lanzar errores y `try/catch/finally` para capturarlos. Lanza siempre instancias de `Error` (o subclases), no strings.

```
function dividir(a, b) {
  if (b === 0) throw new Error("División por cero");
  return a / b;
}

try {
  dividir(10, 0);
} catch (error) {
  console.error(error.message); // "División por cero"
} finally {
  console.log("Operación intentada"); // se ejecuta siempre
}
```

Errores personalizados

```
class ErrorValidacion extends Error {
  constructor(campo) {
    super(`El campo "${campo}" no es válido`);
    this.name = "ErrorValidacion";
    this.campo = campo;
  }
}

try {
  throw new ErrorValidacion("email");
} catch (e) {
  if (e instanceof ErrorValidacion) {
    console.error(`Validación: ${e.campo}`);
  }
}
```

JSON

JSON (JavaScript Object Notation) es el formato estándar de intercambio de datos. Dos métodos clave:

```
const objeto = { nombre: "Sara", activo: true, roles: ["editor"] };

// Objeto → texto JSON
const texto = JSON.stringify(objeto);
// '{"nombre":"Sara","activo":true,"roles":["editor"]}'

const bonito = JSON.stringify(objeto, null, 2); // con indentación de 2 espacios

// Texto JSON → objeto
const recuperado = JSON.parse(texto);
console.log(recuperado.nombre); // "Sara"
```

> ⚠️ Cuidado: `JSON.parse` lanza un error si el texto no es JSON válido. Envuélvelo en `try/catch` cuando los datos vengan de fuera (red, `localStorage`, usuario).

Web APIs del navegador (BOM)

Hasta ahora hemos visto el lenguaje JavaScript "puro". Pero cuando se ejecuta en un navegador, JavaScript dispone además de un enorme catálogo de **Web APIs**: objetos y funciones que el propio navegador pone a su disposición. No forman parte de ECMAScript (no existen en otros entornos como Node), sino que pertenecen al **BOM** (*Browser Object Model*).

El objeto global de todo es `window`, que representa la pestaña del navegador. De él cuelgan, entre muchos otros, `document` (el DOM, Unidad 1), `console`, `fetch`, `localStorage`, `location` o `history`.

Almacenamiento: `localStorage` y `sessionStorage`

Permiten guardar datos en el navegador del usuario como pares clave-valor de **texto**. `localStorage` persiste aunque se cierre el navegador; `sessionStorage` se borra al cerrar la pestaña.

```
// Guardar (solo strings: serializa objetos con JSON)
localStorage.setItem("tema", "oscuro");
localStorage.setItem("usuario", JSON.stringify({ id: 1, nombre: "Ana" }));

// Leer
const tema = localStorage.getItem("tema");                 // "oscuro"
const usuario = JSON.parse(localStorage.getItem("usuario")); // objeto

// Borrar
localStorage.removeItem("tema");
localStorage.clear(); // borra todo
```

> ⚠️ Cuidado: `localStorage` **solo almacena texto**. Para guardar objetos o arrays debes convertirlos con `JSON.stringify` al guardar y `JSON.parse` al leer. Nunca guardes datos sensibles (contraseñas, tokens privados): es accesible desde cualquier JavaScript de la página. Puedes inspeccionarlo en la pestaña *Application* de las DevTools.

Temporizadores

```
// setTimeout: ejecuta una vez tras N milisegundos
const id = setTimeout(() => console.log("3 segundos después"), 3000);
clearTimeout(id); // cancela si aún no se ha ejecutado

// setInterval: ejecuta repetidamente cada N milisegundos
const tic = setInterval(() => console.log("tic"), 1000);
clearInterval(tic); // detiene la repetición
```

`location` e `history`

```
location.href;                // URL completa actual
location.pathname;            // "/productos/42"
location.href = "/inicio";    // navegar a otra página
location.reload();            // recargar

history.back();               // ir atrás (como el botón del navegador)
history.pushState({}, "", "/nueva-ruta"); // cambiar la URL sin recargar (base del
```

> 📌 Nota: `history.pushState` es la pieza que permite a las *Single Page Applications* (como las que harás con React) cambiar de "página" sin recargar. Los enrutadores de React y Next.js se apoyan en esta API.

Otras Web APIs habituales

`fetch` (peticiones HTTP, ya vista y ampliada en la Unidad 4), `navigator` (información del dispositivo, geolocalización, portapapeles), `FormData` (datos de formularios, Unidad 4), `URL` / `URLSearchParams` (manipular URLs y parámetros de consulta), o las APIs de eventos del DOM (`addEventListener`, Unidad 1).

Herramientas modernas: npm, módulos y Vite

Los proyectos reales no se construyen con un único fichero `.js` enlazado a mano. Se apoyan en un **ecosistema de herramientas** que conviene conocer antes de entrar en React (Unidad 3).

npm: el gestor de paquetes

npm (*Node Package Manager*) se instala junto a **Node.js** y permite incorporar librerías de terceros (paquetes) a tu proyecto.

```
npm init -y                # crea un package.json (manifiesto del proyecto)
npm install zod            # instala un paquete y lo añade a dependencies
npm install -D vite        # dependencia solo de desarrollo (-D)
npm run dev                # ejecuta un script definido en package.json
```

- `package.json`: describe el proyecto, sus dependencias y sus *scripts*.
- `node_modules/`: carpeta (enorme) donde npm descarga los paquetes. **Nunca** se sube al repositorio (se ignora con `.gitignore`).
- `package-lock.json`: fija las versiones exactas instaladas para que el proyecto sea reproducible.

> 💡 Consejo: existen alternativas a npm más rápidas y muy usadas en 2026, como **pnpm** y **bun**. Los comandos son casi idénticos (`pnpm install`, `bun install`).

Módulos ES como base del proyecto

Como viste antes, los **módulos ES** (`import` / `export`) permiten organizar el código en ficheros con ámbito propio. Todo proyecto moderno se estructura así: cada componente o utilidad en su fichero, importando lo que necesita. Las herramientas que veremos a continuación entienden esta sintaxis de forma nativa.

Bundlers y servidores de desarrollo: Vite

El navegador no siempre puede consumir directamente decenas de módulos, TypeScript, o ficheros `.jsx` de React. Aquí entran los **bundlers** (empaquetadores), que transforman y combinan tu código fuente en algo que el navegador entiende y sirven todo durante el desarrollo con recarga instantánea (*Hot Module Replacement*).

La herramienta estándar hoy es **Vite**. Su versión **8**, publicada en marzo de 2026, fue el mayor cambio de arquitectura desde Vite 2: unificó en un único empaquetador escrito en Rust, **Rolldown** (estable desde mayo de 2026), lo que antes hacían dos herramientas distintas —esbuild en desarrollo y Rollup en producción—, con compilaciones bastante más rápidas. Es lo que usaremos para crear proyectos de React en la Unidad 3.

```
# Crear un proyecto nuevo con Vite
npm create vite@latest mi-proyecto
cd mi-proyecto
npm install
npm run dev        # servidor de desarrollo en http://localhost:5173
npm run build      # genera la versión optimizada para producción en /dist
```

> 📌 Nota: Vite te preguntará qué *framework* y variante quieres (Vanilla, React, Vue…, con JavaScript o **TypeScript**). Para React elegirás `React` + `TypeScript`, enlazando con la siguiente sección y con la Unidad 3.

Introducción a TypeScript

TypeScript (TS) es un superconjunto de JavaScript creado por Microsoft que añade un **sistema de tipos estáticos**. Todo JavaScript válido es TypeScript válido; TS añade anotaciones que se comprueban **antes** de ejecutar (en tiempo de compilación) y luego "desaparecen": el navegador ejecuta JavaScript normal.

Es prácticamente el **estándar del frontend profesional en 2026**: React, Next.js y la mayoría de proyectos modernos lo usan por defecto. Por eso lo introducimos aquí, antes de React.

> 📌 Nota: **TypeScript 7.0** es estable desde julio de 2026. Es una reescritura del compilador en Go que aprovecha varios hilos y comprueba tipos alrededor de 10× más rápido que la versión anterior. Para lo que verás en este curso no cambia nada: los conceptos del lenguaje (tipos, interfaces, genéricos) son los mismos en TypeScript 5, 6 y 7. Si algún proyecto tuyo depende de herramientas que aún no soportan la versión 7, se puede seguir usando la 6 en paralelo.

¿Por qué tipos?

Detectan errores mientras escribes, en lugar de fallar en tiempo de ejecución ante el usuario:

```
function duplicar(n) {        // JavaScript: acepta cualquier cosa
  return n * 2;
}
duplicar("hola"); // NaN en ejecución, sin avisar 😱
```

```ts
function duplicar(n: number): number { // TypeScript
  return n * 2;
}
duplicar("hola"); // ❌ Error en el editor: 'string' no es asignable a 'number'
```

Tipos básicos

```ts
let nombre: string = "Ana";
let edad: number = 30;
let activo: boolean = true;
let etiquetas: string[] = ["js", "ts"];        // array de strings
let punto: [number, number] = [10, 20];        // tupla

// Unión de tipos: puede ser uno u otro
let id: string | number = 42;

// Inferencia: TS deduce el tipo, no hace falta anotarlo siempre
let ciudad = "Madrid"; // TS sabe que es string
```

Interfaces y tipos para objetos

Para describir la "forma" de un objeto se usan `interface` o `type`. Es muy frecuente para tipar datos y *props* de componentes React.

```ts
interface Usuario {
  id: number;
  nombre: string;
  email?: string;          // opcional (puede no estar)
  readonly creado: Date;   // solo lectura
}

const u: Usuario = { id: 1, nombre: "Lucía", creado: new Date() };

// Tipar funciones
function saludar(usuario: Usuario): string {
  return `Hola, ${usuario.nombre}`;
}
```

Tipos genéricos (idea general)

Permiten escribir código reutilizable que funciona con distintos tipos manteniendo la seguridad. Los verás constantemente en librerías (incluido React: `useState<number>`).

```
function primero<T>(lista: T[]): T | undefined {
  return lista[0];
}
primero<number>([1, 2, 3]); // tipo number
primero(["a", "b"]);        // tipo string (inferido)
```

> 💡 Consejo: no necesitas dominar TypeScript para empezar con React. Bastará con tipar *props*, estado y respuestas del API. La regla práctica: anota lo que cruza fronteras (parámetros de funciones, datos del API) y deja que TS **infiera** el resto.

Buenas prácticas modernas

- **Prefiere la inmutabilidad**: usa `const`, métodos inmutables (`toSorted`, `with`) y *spread* en lugar de mutar.
- **Usa `const` por defecto**, `let` solo si reasignas, nunca `var`.
- **Métodos funcionales** (`map` / `filter` / `reduce`) en vez de bucles manuales cuando aporten claridad.
- `async/await` **con** `try/catch` en vez de cadenas largas de `.then()`.
- **Comprueba** `respuesta.ok` en cada `fetch`.
- **Lanza objetos** `Error`, no strings; crea subclases para errores de dominio.
- `structuredClone` para copias profundas; cuidado con las copias superficiales y los anidados.
- **Desestructura** parámetros y resultados para un código más legible.
- **Funciones puras** siempre que puedas: mismas entradas → mismas salidas, sin efectos secundarios.

Ejercicios propuestos

1. **Inmutabilidad básica.** Dado `const notas = [7, 4, 9, 2]`, obtén un *nuevo* array ordenado de mayor a menor sin modificar `notas`. Verifica con `console.log` que el original permanece intacto. (Pista: `toSorted`.)

2. **Pipeline funcional.** Partiendo del array `productos` del apartado de objetos, escribe una sola expresión encadenada que devuelva los nombres (en mayúsculas y ordenados alfabéticamente) de los productos con `stock > 0`.
3. **Agrupar con `Object.groupBy`.** Dado un array de alumnos `{ nombre, nota }`, agrúpalos en `{ aprobados: [...], suspensos: [...] }` según si la nota es `>= 5`.
4. **Copia profunda.** Crea un objeto `config` con al menos un objeto anidado. Haz una copia con *spread* y otra con `structuredClone`. Modifica el objeto anidado en cada copia y explica con `console.log` por qué los resultados difieren.
5. **Closure contador.** Implementa una *factory* `crearMonedero(saldoInicial)` que devuelva métodos `ingresar(n)`, `retirar(n)` y `consultar()`. El saldo debe ser privado (no accesible directamente desde fuera) y `retirar` no debe permitir saldo negativo (lanza un `Error`).
6. **Clases y herencia.** Define una clase `Figura` con un método `area()` que lance un error ("no implementado"). Crea las subclases `Circulo` y `Rectangulo` que sobreescriban `area()`. Crea un array de figuras y muestra el área total con `reduce`.
7. **`Map` y `Set`.** A partir de un array de palabras, usa un `Map` para contar cuántas veces aparece cada una y un `Set` para listar las palabras únicas.
8. **Asincronía y orden.** Sin ejecutarlo, predice el orden de salida del siguiente código y luego compruébalo: combina `console.log` síncronos, un `setTimeout(…, 0)` y un `Promise.resolve().then(…)`. Explica el resultado con el modelo del event loop.
9. **`fetch` con manejo de errores.** Usando la API pública `https://jsonplaceholder.typicode.com/users`, escribe una función `async` que obtenga los usuarios, compruebe `respuesta.ok`, devuelva un array con sus nombres y maneje los errores devolviendo `[]`.
10. **Combinadores de promesas.** Crea tres funciones que devuelvan promesas con `setTimeout` de distinta duración. Compara el comportamiento de `Promise.all`, `Promise.allSettled` y `Promise.race` cuando una de ellas rechaza.
11. **Preferencias con `localStorage`.** Crea una página con un botón que alterne entre tema claro y oscuro (cambiando una clase del `<body>`). Guarda la preferencia en `localStorage` y aplícala automáticamente al recargar. Comprueba el valor guardado en la pestaña *Application* de las DevTools.
12. **Tu primer proyecto con Vite.** Crea un proyecto con `npm create vite@latest` (variante *Vanilla* + *TypeScript*), instala dependencias y arráncalo con `npm run dev`. Identifica el `package.json`, los *scripts* disponibles y por qué `node_modules/` no debe subirse al repositorio.

13. **De JavaScript a TypeScript.** Toma una función JS sencilla (por ejemplo, la del ejercicio 5 sobre el monedero) y añádele tipos: anota parámetros, valor de retorno y define una `interface` para el objeto devuelto. Provoca a propósito un error de tipo y observa cómo lo señala el editor.

Recursos y referencias

- MDN — Array (métodos): https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array
- MDN — `Object.groupBy`: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/groupBy
- MDN — Desestructuración: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment
- MDN — `this`: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Operators/this
- MDN — Closures: https://developer.mozilla.org/es/docs/Web/JavaScript/Closures
- MDN — Clases: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes
- MDN — `Map` y `Set`: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Map
- MDN — Promesas: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Promise
- MDN — `async` / `await`: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/async_function
- MDN — `fetch`: https://developer.mozilla.org/es/docs/Web/API/Window/fetch
- MDN — Módulos JavaScript: https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Modules
- MDN — `structuredClone`: https://developer.mozilla.org/es/docs/Web/API/structuredClone
- javascript.info — El event loop y microtareas: https://javascript.info/event-loop
- MDN — Web Storage (`localStorage`): https://developer.mozilla.org/es/docs/Web/API/Web_Storage_API
- MDN — `setTimeout` / `setInterval`: https://developer.mozilla.org/es/docs/Web/API/setTimeout
- Documentación oficial de npm: https://docs.npmjs.com/
- Vite — Guía de inicio: https://vite.dev/guide/
- TypeScript — Manual oficial: https://www.typescriptlang.org/docs/handbook/intro.html

- TypeScript para programadores de JavaScript: https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html